

Politique de contrôle d'accès — Horizon

Organisation : Vibe Coding Mind **Application** : Horizon (gestion de finances personnelles pour couples) **Responsable de la sécurité et de la protection des renseignements personnels** : Sébastien Normandeau — sebastiennormandeau@gmail.com **Version** : 1.0 — 19 juillet 2026 · **Prochaine revue** : octobre 2026 (revue trimestrielle)

Cette politique complète [SECURITY_POLICY.md](#) §1 en détaillant les rôles, les niveaux d'accès et les procédures de gestion des accès pour l'ensemble des systèmes d'Horizon.

1. Principe directeur : moindre privilège

Chaque identité — utilisateur, administrateur ou compte de service — ne reçoit que l'accès strictement nécessaire à sa fonction, et rien de plus. Ce principe est appliqué à trois niveaux :

1. **Par conception** : l'architecture serveur-autoritaire d'Horizon fait qu'aucun rôle client, même compromis, ne peut écrire les champs financiers directement (voir §3).
2. **Par défaut** : les nouveaux accès sont accordés au niveau le plus restrictif qui permette d'accomplir la tâche.
3. **Par révision** : les accès sont revus trimestriellement (§5) plutôt que laissés à s'accumuler.

2. Rôles et niveaux d'accès

A. Propriétaire / administrateur

Qui : Sébastien Normandeau — seule personne avec ce rôle actuellement.

Accès détenus : - Compte Google propriétaire du projet Firebase/Google Cloud horizon-dbba0 (rôle IAM Owner) — contrôle Firestore, Cloud Functions, Secret Manager, Identity Platform, facturation. - Dépôt GitHub du code source (accès complet, y compris déploiement). - Consoles des fournisseurs tiers : Plaid, RevenueCat (à la création), Anthropic. - Script d'administration MFA local (functions/scripts/mfa-admin.js), utilisable uniquement avec ces mêmes identifiants Google.

Contrôles obligatoires : authentification multifacteur sur chacun de ces comptes (voir PRODUCTION_CHECKLIST.md §0), poste de travail chiffré (SECURITY_POLICY.md §5).

Ce rôle n'est PAS accordé aux utilisateurs finaux : aucun utilisateur de l'application, même dans un foyer, n'a d'accès à ces consoles.

B. Utilisateur final (membre d'un foyer)

Qui : toute personne ayant créé un compte dans l'application.

Accès détenus — appliqués par `firestore.rules`, jamais par simple convention côté client : - Son propre document `users/{uid}` : lecture/écriture, à l'exception des champs `household_id`, `role` et `subscription_tier` qui restent en écriture exclusive aux Cloud Functions. - Le document `households/{id}` de son propre foyer (calculé côté serveur via `userHouseholdId()`) : lecture, et écriture limitée — `join_code`, `user_A_id`, `user_B_id`, `created_by`, `created_at` et `subscription_tier` sont protégés contre toute modification cliente. - Les transactions de son foyer : lecture, et écriture strictement limitée à deux champs, `assigned_to_bucket` et `category` — jamais le montant, le commerçant ou l'identité de la transaction. - **Aucun accès** aux foyers d'autres utilisateurs : la règle `isUserInHousehold()` compare le `household_id` de la ressource à celui du profil de l'appelant, recalculé côté serveur à chaque requête. - **Aucun accès, en lecture ni en écriture**, aux jetons bancaires (`bank_connections`) ni aux compteurs de limitation de débit (`rate_limits`) : ces collections sont en interdiction totale (`allow read, write: if false`), accessibles uniquement via l'Admin SDK des Cloud Functions. - **Double authentification (TOTP) obligatoire** avant tout accès aux données du foyer (voir `SECURITY_POLICY.md` §1 et §6 bis).

C. Comptes de service (Cloud Functions)

Qui : les fonctions serveur elles-mêmes, exécutées avec les identifiants de service du projet Firebase.

Accès détenus : les Cloud Functions utilisent l'**Admin SDK**, qui contourne intégralement les règles Firestore — c'est le mécanisme par lequel le serveur agit comme seule autorité sur les champs financiers (écriture de `safe_to_spend_*`, `internal_debt_balance`, `subscription_tier`, gestion de `bank_connections`, etc.). Chaque fonction callable applique néanmoins ses propres contrôles avant d'agir : - `requireAuth()` : exige une session authentifiée et, sauf exception explicite, un courriel vérifié ; applique aussi la vérification App Check quand `ENFORCE_APP_CHECK=true`. - `enforceRateLimit()` : limite le nombre d'appels par utilisateur et par fonction dans une fenêtre glissante, pour contenir l'impact d'un jeton compromis ou d'un abus. - `assertString()` et motifs regex dédiés : aucune entrée n'atteint la logique métier sans validation de type, de longueur et de format.

Secrets : les identifiants des fournisseurs tiers (Plaid, RevenueCat, Anthropic) résident exclusivement dans **Google Secret Manager** et sont injectés aux fonctions par `runWith({ secrets: [...] })` — jamais dans le code, jamais dans `functions/.env` (voir `SECURITY_POLICY.md` §3).

D. Fournisseurs tiers (accès délégué, jamais direct)

Plaid, RevenueCat et Anthropic ne reçoivent jamais d'accès direct à Firestore. L'échange de données passe exclusivement par des appels API sortants initiés par les Cloud Functions, ou par des webhooks entrants authentifiés (signature JWT ES256 pour Plaid, secret d'autorisation dédié pour RevenueCat) et traités par une fonction serveur qui applique les mêmes contrôles que ci-dessus avant d'écrire quoi que ce soit.

3. Contrôles techniques en place (résumé)

Contrôle	Où	Effet
Isolation par foyer	<code>firestore.rules</code>	Un utilisateur ne voit jamais

Contrôle	Où	Effet
Champs financiers protégés	<code>firestore.rules</code> (<code>transactions</code> , <code>households</code>)	les données d'un autre foyer Le client ne peut écrire que <code>assigned_to_bucket / category</code> sur une transaction ; les champs d'identité du foyer sont verrouillés
Jetons bancaires hors de portée	<code>firestore.rules</code> (<code>bank_connections</code> : <code>if false</code>)	Aucune lecture ni écriture cliente possible, en toute circonstance
Compteurs anti-abus hors de portée	<code>firestore.rules</code> (<code>rate_limits</code> : <code>if false</code>)	Un client ne peut ni lire ni manipuler ses propres limites
Authentification obligatoire	<code>security.ts</code> – <code>requireAuth()</code>	Aucune fonction appellable n'agit sans session valide
Courriel vérifié obligatoire	<code>security.ts</code> , <code>firestore.rules</code> , <code>AuthRouter</code> (client)	Triple couche, redondante par conception
MFA obligatoire	Identity Platform (serveur) + porte bloquante dans l'app	Aucun accès aux données sans second facteur enrôlé
Limitation de débit	<code>security.ts</code> – <code>enforceRateLimit()</code>	Contient l'impact d'un jeton ou d'un compte compromis
Validation des entrées	<code>security.ts</code> – <code>assertString()</code> + motifs	Aucune donnée non conforme n'atteint la logique métier
Secrets hors du code	Google Secret Manager	Aucun identifiant tiers dans le dépôt, les journaux ou le client
Anti-bot	Firebase App Check (<code>ENFORCE_APP_CHECK</code>)	Rejette les appels sans attestation d'appareil valide, une fois activé

4. Octroi et retrait d'accès

Octroi

Tout nouvel accès (compte administrateur, dépôt, console tierce, secret) suit ces étapes avant d'être activé :

1. **Justifier le besoin** : quelle tâche exige cet accès, et à quel niveau minimal.
2. **Choisir le rôle le plus restrictif** qui permette d'accomplir cette tâche (par exemple, un rôle Firebase « Viewer » plutôt que « Owner » pour une personne qui n'a besoin que de consulter les journaux).

3. **Activer le MFA** sur le nouveau compte avant sa première utilisation réelle — non négociable pour tout accès administrateur.
4. **Consigner** l'octroi : date, personne, accès accordé, justification.

Retrait

Tout accès est retiré **immédiatement** dans les cas suivants : - Fin d'une collaboration ou d'un contrat. - Suspicion de compromission d'un compte ou d'un appareil. - Accès devenu inutile (tâche terminée, rôle changé).

Le retrait couvre : révocation des rôles IAM, retrait du dépôt GitHub, rotation des secrets partagés (Secret Manager), et — pour un poste de travail perdu ou volé — révocation des identifiants locaux (`gcloud auth application-default revoke`, `firebase logout`), conformément à `SECURITY_POLICY.md` §5.

5. Revue périodique des accès

Les accès sont revus **au moins une fois par trimestre** par le responsable désigné. La revue vérifie :

- La liste des comptes ayant un accès administrateur au projet Firebase correspond exactement aux personnes qui en ont réellement besoin aujourd'hui.
- Chaque secret dans Google Secret Manager est encore utilisé par une fonction active ; les secrets orphelins sont supprimés.
- Le MFA reste actif sur tous les comptes administrateur et sur les comptes utilisateurs enrôlés.
- Aucun accès temporaire accordé dans le trimestre n'a été oublié en place au-delà de sa justification initiale.

Chaque revue est datée et ses constats (rien à signaler, ou accès retirés) sont consignés. La date de la prochaine revue est mise à jour en tête de ce document à l'issue de chacune.

6. Portée et limites actuelles

Horizon est aujourd'hui exploité par un studio solo (Vibe Coding Mind, une seule personne avec accès administrateur). Cette politique est écrite pour rester valide à mesure que l'équipe grandit : l'ajout d'une deuxième personne avec accès administrateur, d'un contractuel, ou d'un rôle de soutien à la clientèle devra suivre la procédure d'octroi du §4 et apparaître dans la revue trimestrielle du §5 — pas être ajouté de manière informelle.

7. Revue de la politique

Cette politique est revue **au moins une fois par trimestre**, en même temps que la revue des accès (§5), ainsi qu'après tout changement significatif de rôles ou d'architecture. Chaque revue met à jour la version et la date en tête du document.